11.21 Will the following code cause an error, or will it compile without any errors? If it causes an error, rewrite it so it compiles.

```
enum Color { RED, GREEN, BLUE };
Color c = RED;
c++;
```

NOTE: For an additional example of this chapter's topics, see the High Adventure Travel Part 2 Case Study on the Computer Science Portal at www.pearsonglobaleditions.com/gaddis.

Review Questions and Exercises

Short Answer

1. What is a primitive data type?
2. Does a structure declaration cause a structure variable to be created?
3. Both arrays and structures are capable of storing multiple values. What is the difference between an array and a structure?
4. Look at the following structure declaration:

```
struct Point
{
   int x;
   int y;
};
```

Write statements that

A) define a `Point` structure variable named `center`.
B) assign 12 to the `x` member of `center`.
C) assign 7 to the `y` member of `center`.
D) display the contents of the `x` and `y` members of `center`.

5. Look at the following structure declaration:

```
struct FullName
{
   string lastName;
   string middleName;
   string firstName;
};
```

Write statements that

A) define a `FullName` structure variable named `info`.
B) assign your last, middle, and first name to the members of the `info` variable.
C) display the contents of the members of the `info` variable.

6. Look at the following code:

```
struct PartData
{
   string partName;
   int idNumber;
};
PartData inventory[100];
```

Write a statement that displays the contents of the `partName` member of element 49 of the `inventory` array.

7. Look at the following code:

```
struct Town
{
   string townName;
   string countyName;
   double population;
   double elevation;
};

Town t = { "Canton", "Haywood", 9478 };
```

A) What value is stored in `t.townName`?
B) What value is stored in `t.countyName`?
C) What value is stored in `t.population`?
D) What value is stored in `t.elevation`?

8. Look at the following code:

```
structure Rectangle
{
   int length;
   int width;
};
Rectangle *r = nullptr
```

Write statements that

A) dynamically allocate a `Rectangle` structure variable and use `r` to point to it.
B) assign 10 to the structure's `length` member and 14 to the structure's `width` member.

9. Which declaration will you prefer from the following and why?

```
enum TruthVal { TRUE, FALSE };
enum TruthVal { FALSE, TRUE };
```

10. Look at the following declaration:

```
enum Person { BILL, JOHN, CLAIRE, BOB };
Person p;
```

Indicate whether each of the following statements or expressions is valid or invalid.

A) `p = BOB;`
B) `p++;`
C) `BILL > BOB`
D) `p = 0;`
E) `int x = BILL;`
F) `p = static_cast<Person>(3);`
G) `cout << CLAIRE << endl;`

Fill-in-the-Blank

11. Before a structure variable can be created, the structure must be _____.
12. When accessing a structure member, the variable to the left of the dot operator is _____.
13. A(n) _____ data type does not have a name.
14. The data members of a(n) _____ occupy the same memory location.
15. In the definition of a structure variable, the _____ is placed before the variable name, just like the data type of a regular variable is placed before its name.
16. The _____ operator is used to access structure members from a structure variable.

Algorithm Workbench

17. The structure `Car` is declared as follows:

```
struct Car
{
   string carMake;
   string carModel;
   int yearModel;
   double cost;
};
```

Write a definition statement that defines a `Car` structure variable initialized with the following data:

Make: Ford
Model: Mustang
Year Model: 1968
Cost: $20,000

18. Define an array of 25 of the `Car` structure variables (the structure is declared in Question 17).
19. Define an array of 35 of the `Car` structure variables. Initialize the first three elements with the following data:

Make	Model	Year	Cost
Ford	Taurus	1997	$21,000
Honda	Accord	1992	$11,000
Lamborghini	Countach	1997	$200,000

20. Write a loop that will step through the array you defined in Question 19, displaying the contents of each element.
21. Declare a structure named `TempScale`, with the following members:

```
fahrenheit: a double
centigrade: a double
```

Next, declare a structure named `Reading`, with the following members:

```
windSpeed: an int
humidity: a double
temperature: a TempScale structure variable
```

Next, define a `Reading` structure variable.

22. Write statements that will store the following data in the variable you defined in Problem 21:

    Wind Speed: 37 mph
    Humidity: 32%
    Fahrenheit temperature: 32 degrees
    Centigrade temperature: 0 degrees

23. Write a function called `showReading`. It should accept a `Reading` structure variable (see Problem 21) as its argument. The function should display the contents of the variable on the screen.

24. Write a function called `findReading`. It should use a `Reading` structure reference variable (see Problem 21) as its parameter. The function should ask the user to enter values for each member of the structure.

25. Write a function called `getReading`, which returns a `Reading` structure (see Problem 21). The function should ask the user to enter values for each member of a `Reading` structure, then return the structure.

26. Write a function called `recordReading`. It should use a `Reading` structure pointer variable (see Problem 21) as its parameter. The function should ask the user to enter values for each member of the structure pointed to by the parameter.

27. What is the purpose of the following expression that has used nested structures?

```
employee.address.postcode = 9909;
```

28. Look at the following statement:

```
enum Shape3D { SPHERE, CUBE, CYLINDER };
```

    A) What are the enumerators and what are their default values?
    B) How can you assign values of the enumerators to the number of surfaces that it has?
    C) Write code to declare three `Shape3D` variables `item1, item2, and item3,` and assign them the permissible values.

29. Declare an enumerated data type `Shape` that has three enumerators `CIRCLE, SQUARE,` and `TRIANGLE`, and one `Shape` variable in a single statement.

True or False

30. T F A semicolon is required after the closing brace of a structure declaration.
31. T F Abstract data types are defined as the basic parts of a computer language.
32. T F The contents of a structure variable can be displayed by passing the structure variable to the `cout` object.
33. T F Structure member variables may be initialized in the declaration of the structure.
34. T F In a structure variable's initialization list, you do not have to provide initializers for all the members.
35. T F A structure and a union can be used interchangeably.
36. T F The following expression refers to element 5 in the array `carInfo`: `carInfo.model[5]`
37. T F It is possible to dynamically allocate an array of structures.

38. T F A structure definition allocates space in memory for all its members.
39. T F Nested structure implies a structure variable within another structured variable.
40. T F An entire structure may not be passed to a function as an argument.
41. T F A function may return a pointer to a structure variable.
42. T F When a function returns a structure, it is always necessary for the function to have a local structure variable to hold the member values that are to be returned.
43. T F The indirection operator has higher precedence than the dot operator.
44. T F The structure pointer operator does not automatically dereference the structure pointer on its left.

Find the Errors

Each of the following declarations, programs, and program segments has errors. Locate as many as you can.

45.
```
Struct myStruct
{
   int x;
   float y;
};
```

46.
```
struct Lists
{
   int number[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
   int total;
}
```

47.
```
struct TwoVals
{
   int a, b;
};
int main ()
{
   TwoVals.a = 10;
   TwoVals.b = 20;
   return 0;
}
```

48.
```
#include <iostream>
using namespace std;

struct ThreeVals
{
   int a, b, c;
};
int main()
{
   ThreeVals vals = {1, 2, 3};
   cout << vals << endl;
   return 0;
}
```

49.
```
#include <iostream>
#include <string>
using namespace std;

struct names
{
   string first;
   string last;
};
int main ()
{
   names customer = "Smith", "Orley";
   cout << names.first << endl;
   cout << names.last << endl;
   return 0;
}
```

50.
```
struct FourVals
{
   int a, b, c, d;
};
int main ()
{
   FourVals nums = {1, 2, , 4};
   return 0;
}
```

51.
```
struct TwoVals
{
   int a;
   int b;
};
TwoVals getVals()
{
   TwoVals.a = TwoVals.b = 0;
}
```

52.
```
struct ThreeVals
{
   int a, b, c;
};
int main ()
{
   TwoVals s, *sptr = nullptr;
   sptr = &s;
   *sptr.a = 1;
   return 0;
}
```